Neurocomputing 638 (2025) 130132

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Multi-resolution Patch-based Fourier Graph Spectral Network for spatiotemporal time series forecasting

Jiankai Zheng [a], Liang Xie [a,*], Haijiao Xu [b]

[a] School of Mathematics and Statistics, Wuhan University of Technology, Wuhan, 430070, China
[b] School of Computer Science, Guangdong University of Education, Guangzhou, 510303, China

ARTICLE INFO

Communicated by F.G. Guimaraes

Keywords:
Spatiotemporal time series
Multi-resolution patch
Hyperpatch graph
Fourier graph spectral network
Residual fusion mechanism

ABSTRACT

Spatiotemporal time series is of great significance in many fields such as finance and meteorology. Accurate forecasting depends on the effective understanding of their spatiotemporal correlations. However, existing methods have two major problems: firstly, the analysis of multi-resolution spatiotemporal correlations is insufficient; secondly, they are difficult to analyze in terms of spatiotemporal correlations efficiently. Some graph-based methods have high computational costs, while efficient methods struggle to learn spatiotemporal relationships. To overcome these problems, we introduce the Multi-resolution Patch-based Fourier Graph Spectral Network (MPFGSN), which leverages a hyperpatch graph and a novel graph spectral convolution to spatiotemporal time series prediction, enhancing temporal dependency learning. We first capture the multi-scale features of the time series through multi-resolution patches, then represent the spatiotemporal relations in the time series with the hyperpatch graph structure. Then, the Fourier Graph Spectral Network (FGSN) is proposed. The FGSN utilizes Fourier transformation and point multiplication operations in the spectral domain to implement efficient graph spectral convolution. And it maintains the capabilities of traditional multi-layer graph convolution while also being faster. Finally, it integrates information from different resolutions through a residual fusion mechanism, capturing both long-term trends and short-term fluctuations, thus enhancing the model's robustness. Extensive experiments on public datasets confirm MPFGSN's superiority in accuracy and generalization over existing models, validating its practical applicability in sophisticated forecasting scenarios.

1. Introduction

Spatiotemporal time series forecasting is crucial for fields such as transportation, finance, and disease surveillance. These datasets capture spatial patterns that evolve over time, highlighting the progression of temporal trends and spatial interdependencies. In transportation and disease surveillance, datasets reveal the interplay between different geographic locations, which is crucial for understanding traffic flow and disease propagation patterns. For instance, in transportation, time series data not only reflect how traffic volumes at different locations change over time but also reveal the interdependencies between geographic locations. In the field of disease surveillance, datasets track how diseases spread across different regions over time, demonstrating the importance of spatial distribution and transmission routes. In finance, datasets show how prices of different stocks or exchange rates between countries change over time, reflecting the interactions within financial markets [1–3]. The in-depth analysis of these spatiotemporal correlations is essential for enhancing predictive accuracy, aiding in the more precise forecasting of traffic flows, financial market trends, and disease propagation patterns. Despite various time series forecasting architectures proposed in existing research, such as Convolutional Neural Networks (CNNs) [4], Recurrent Neural Networks (RNNs) [5], and the latest Transformer-based methods [6,7], there are still many challenges in modeling spatiotemporal correlations and multi-resolution feature analysis.

In the field of time series forecasting, methods based on Graph Neural Networks (GNNs) have shown significant performance improvements [8–10]. These methods rely on the combination of graph networks, such as Graph Convolutional Networks (GCNs) [11], and temporal networks, such as Long Short-Term Memory (LSTM) [12], to capture the spatial dynamics between sequences and the temporal dependencies within sequences. However, the separated spatiotemporal modeling approach has not fully considered the integrated relationship between space and time as they exist in the real world, which constrains the enhancement of forecasting accuracy. To address this issue, Fourier Graph Neural Networks (FourierGNN) [13] have been proposed, which re-examine the time series forecasting problem from the perspective of

* Corresponding author.
E-mail addresses: 312796@whut.edu.cn (J. Zheng), whutxl@hotmail.com (L. Xie), guesskkk@alumni.hust.edu.cn (H. Xu).

https://doi.org/10.1016/j.neucom.2025.130132
Received 22 November 2024; Received in revised form 28 February 2025; Accepted 24 March 2025
Available online 3 April 2025

graph structure. However, FourierGNN and its related works [14–16] have not considered patches of multiple resolutions, resulting in the inability to fully leverage the information available at various levels of data resolution [17].

Multi-resolution data [18–20] provides additional valuable information for time series forecasting. For example, in financial investment decision-making, analysts will consider stock data at different resolutions such as weekly, daily, and minutely. Long-term data features reveal overall trends, while short-term data features show subtle changes over a short period, both of which are extremely important for improving the quality of forecasts. In addition, recent studies [21,22] demonstrate that timestamp-level tokens limit the effectiveness of the time series model in capturing temporal patterns. In contrast, patch-level tokens (where a patch is a time step window) enable the time series model to model temporal patterns within each patch and learn the relationships between different patches [23].

Although existing methods have shown promising results in time series forecasting, there are still two main issues that need to be addressed. Firstly, the analysis of multi-resolution spatiotemporal correlations is a challenge that neither current multi-resolution methods nor spatiotemporal approaches like FourierGNN have fully considered. Secondly, the complexity of spatiotemporal correlation analysis poses a significant challenge in terms of computational efficiency. Many existing graph-based methods are computationally intensive. Other methods such as MLP-mixer [20] and FourierGNN [13] are efficient in computation, but they fail to effectively learn the spatiotemporal relationship.

In response to these challenges, we propose a Multi-Resolution Patch-based Fourier Spectral Graph Network (MPFGSN). The model decomposes time series data into multiple time windows (patches) and uses a hyperpatch graph data structure to capture the relations of different variables at different time periods under different resolutions. MPFGSN not only captures spatiotemporal relationships but also significantly improves the effects of long-sequence forecasting. Our Fourier Graph Spectral Network (FGSN) achieves efficient graph spectral convolution by applying Fourier transformation and point-wise multiplication in the spectral domain. The theoretical analysis confirms that FGSN retains the performance of traditional multi-layer graph convolution and has a significant advantage in speed. By deepening the understanding and prediction of complex spatiotemporal changes in time series data, FGSN successfully overcomes the limitations of traditional forecasting methods. In addition, by introducing multi-resolution patches and residual fusion mechanisms, MPFGSN not only reduces computational costs but also integrates information under different time resolutions and avoids information redundancy. With these advanced approaches, the MPFGSN model shows its advantages in dealing with complex spatiotemporal dependencies, especially in the field of time series forecasting.

In summary, our main contributions are:

1. Innovative FGSN Model: We propose an innovative FGSN model that incorporates spectral domain graph convolution, which is theoretically equivalent to general graph convolution in terms of effectiveness but offers greater computational efficiency. This approach effectively captures the complex spatiotemporal patterns within spatiotemporal time series data, resulting in a model with lower time complexity and strong adaptive learning capabilities. These attributes make it particularly suitable for large-scale graph structure data, thereby enhancing the accuracy and efficiency of time series forecasting.
2. Multi-Resolution Patch and Residual Fusion Mechanism: We introduce a multi-resolution patch mechanism that employs a hyperpatch graph data structure. The hyperpatch graph integrates spatiotemporal information between patches into a fully connected graph, effectively capturing the dynamic relations of patches across different timescales. Additionally, we incorporate a residual fusion mechanism to prevent information redundancy. This mechanism adeptly captures both the long-term trends and short-term fluctuations inherent in time series.
3. Experimental Validation: We conduct comprehensive experiments across multiple public time series datasets. Our method integrates a Patch-based FGSN model with a multi-resolution patch and residual fusion mechanism, enhancing its forecasting accuracy and generalization ability. The comprehensive experiments across multiple public time series datasets confirm its superiority, outperforming the existing methods for spatiotemporal time series forecasting.

The remainder of this paper is organized as follows: First, Section 2 reviews the related work, followed by Section 3, which introduces the problem definition. Section 4 elaborates on the model framework, Section 5 presents the experimental results, and Section 6 concludes the paper.

2. Related work

Spatiotemporal time series forecasting is widely applied in multiple fields. Although numerous related methods have emerged, they still have limitations. In this section, we will focus on existing methods, including patch-based methods and the application of graph neural networks in multivariate time series forecasting. We will analyze their advantages and disadvantages to lay the foundation for the research of the MPFGSN model.

2.1. Spatiotemporal time series forecasting

Multivariate time series forecasting predicts future values by modeling multiple related variables. Patch-based methods (such as PatchTST [22], MTST [24]) divide time series segments with the help of the Transformer architecture, effectively capturing temporal dependencies; methods based on time series decomposition (such as TimesNet [25], TimeMixer [26]) enhance the performance of long-sequence forecasting by optimizing the decomposition process, accurately grasping the internal temporal structure of the sequence.

In the realm of multivariate time series forecasting, spatiotemporal forecasting methods go a step further compared to traditional methods by additionally considering the spatial correlations among variables. These spatiotemporal methods have found extensive applications across numerous fields [27,28] and play a pivotal role in time series forecasting [29]. STPGNN [30] identifies key nodes to capture complex spatiotemporal dependencies and optimize traffic forecasting; MiTSformer [6] recovers latent continuous variables and utilizes multi-scale context for balanced modeling; GPT-ST [31] introduces a spatiotemporal pre-training framework to enhance the learning ability of downstream models. These methods provide different solutions for spatiotemporal time series forecasting.

However, existing spatiotemporal methods have drawbacks. Some methods have weak capabilities in fusing information of different resolutions when dealing with high-resolution data, and it is difficult to capture complex spatiotemporal patterns. Moreover, many methods have low computational efficiency, mainly due to the complex operations of the spatial processing module. Traditional spatial domain graph convolution operations are computationally intensive when processing large-scale data and cannot meet real-time requirements.

In contrast, the MPFGSN model has significant advantages. Its multi-resolution patch mechanism integrates information from different time scales, accurately capturing spatiotemporal patterns. The hyperpatch graph structure and FGSN improve the processing ability of large-scale graph data and computational efficiency. The residual fusion mechanism eliminates information redundancy and enhances the generalization ability, providing a better solution for spatiotemporal time series forecasting.

2.2. Patch-based methods

In NLP, the BERT model has shown success by processing text data through subword tokenization [32], and in CV, the Vision Transformer (ViT) handles image data by segmenting images into patches [33]. The success of these methods indicates that patch-based representations can significantly improve model performance. In time series forecasting, patch-based methods have demonstrated their effectiveness in capturing temporal patterns. A patch can be seen as a continuous time window in time series data, allowing the model to learn local patterns within each time window and understand the relationships between different time windows. For instance, researches by Zhang and Yan (2023) [21] and Nie et al. (2023) [22] have shown that compared to timestamp-level tokens, patch-level tokens can help models to more effectively capture temporal patterns in time series data. The advantage of this approach is that it retains local semantic information in tokens, reduces the computational and memory usage of attention maps, and allows the model to focus on longer histories. The introduction of patch-level tokens provides a new perspective and direction for improvement in spatiotemporal time series forecasting. Therefore, we propose a multi-resolution patch mechanism that can integrate information from different time scales, improving the accuracy and computational efficiency of time series forecasting, while also retaining good generalization and adaptive learning potential.

2.3. Graph neural networks for multivariate time series forecasting

The wide application of GNNs in spatiotemporal time series forecasting is primarily due to their ability to effectively model structural dependencies between variables. Models such as STGCN [34], DCRNN [35], and TAMP-S2GCNets [10] utilize GNNs to process spatiotemporal time series data. These models typically require a predefined graph structure, which is often unknown in practical applications. To address this limitation, some studies automatically learn the graph structure through inter-sequence correlations, such as node similarity [36,37] or self-attention mechanisms [8]. These methods use graph networks to handle spatial correlations and learn time dependencies through temporal networks. FourierGNN [13] achieves spatiotemporal time series forecasting from a purely graphical perspective, capable of spatiotemporal relationships simultaneously.

Graph convolutional methods effectively capture complex spatiotemporal relationships in time series data by simulating structural dependencies among variables. Spatial domain graph convolution is a technique used in graph neural networks to capture the features of the local neighborhood of nodes, where GCNs [11] updates the representation of each node by aggregating the features of neighboring nodes, and MPNN [38] achieves this process through message passing and node updating mechanisms, but spatial domain methods may face limitations in computational efficiency and scalability, especially on large graph data. Spectral-domain graph convolution is a convolution operation conducted in the spectral domain based on the graph's Laplacian matrix. SCNN [39] applies convolution directly in the frequency domain, and ChebNet [40] uses Chebyshev polynomials to approximate eigenvalue decomposition, both of which can effectively capture the global and local structures of the graph. However, existing methods based on spectral domain graph convolution and its improvements do not utilize the characteristics of graph spectral transformation to simplify convolution operations, resulting in higher time complexity.

The proposed MPFGSN model automatically learns and captures complex dependencies between variables through the hyperpatch graph data structure, thus overcoming the limitations of pre-defined graph structures in traditional methods. By employing spectral domain convolution based on the Discrete Fourier Transform (DFT), the MPFGSN model reduces the step of solving eigenvalues and significantly lowers computational costs, providing a more efficient solution for spatiotemporal time series forecasting.

3. Problem definition

The problem of spatiotemporal time series forecasting can be defined as follows: given a dataset containing multiple variables, each with observations at consecutive time points. The objective is to predict the future values of each variable at several time points based on past observations.

Specifically, let us assume we have a time series dataset composed of N variables, each with observations recorded at T consecutive time points. These observations can be organized into an $T \times N$ matrix X_t, where each column represents the observations of a single variable across the T time points. For timestamp t, there is an observation window $X_t = [x_{t-T+1}, \dots, x_t]$ of length T; our goal is to forecast the values of the individual variables at $\mathcal{T}_{Pred}$ time points from time $t+1$ to $t+\mathcal{T}_{Pred}$, where $\mathcal{T}_{Pred}$ is the length.

Formally, the forecasting task is to find function F that uses current data X_t to predict future values $Y_t = [x_{t+1}, \dots, x_{t+\mathcal{T}_{Pred}}] \in \mathbb{R}^{T \times N}$ for $\mathcal{T}_{Pred}$ time points. This function is parameterized by θ, representing the learnable part of the model. The forecasting model can be represented as:

$$\hat{Y}_t = F_\theta(X_t) \tag{1}$$

where $\hat{Y}_t$ is the model's predicted value for a future time point $t+1, \dots, t+\mathcal{T}_{Pred}$, and F_θ is a prediction function based on the parameter θ that captures complex patterns and dependencies in time series data.

4. Methodology

4.1. Framework

Fig. 1 illustrates the framework of the MPFGSN model, which consists of the following three core components: (a) Multi-resolution Patches: This component creates subsequences (patches) of the time series at different temporal resolutions, allowing the model to capture features of the time series at various resolutions and thereby enriching the data representation at a multi-resolution level. (b) FGSN: The core of this module lies in its spectral convolution structure, which leverages the theoretical foundation of the Fourier transform to simulate the effect of graph convolution through point-wise multiplication in the spectral domain, effectively enabling the model to learn the periodicity and spatiotemporal dynamics of time series. (c) Residual Fusion: This part computes the residuals between patches of different resolutions to eliminate information redundancy, ensuring that each part can identify features specific to certain resolutions, thus achieving effective information fusion across temporal resolutions.

4.2. Multi-resolution patch

Given a spatiotemporal time series window, the input at each layer is N variables at timestamp t, denoted as $X_t \in \mathbb{R}^{T \times N}$. Let p^m represent the patch size corresponding to the mth layer.

Then, input X_t into the $p^m - patching$ module $\mathcal{T}^m$ from $\mathbb{R}^{T \times N}$ to $\mathbb{R}^{J^m \times p^m \times N}$. We describe the patching process in Fig. 2, where all patches do not overlap. This module transforms X_t into $J^m = \lfloor T/p^m \rfloor$ patches. The specific calculation of $\hat{X}_t^m = [\hat{X}_{t,1}^m, \hat{X}_{t,2}^m, \dots, \hat{X}_{t,J^m}^m] = \mathcal{T}^m(X_t)$ is as follows:

$$\hat{X}_{t,j}^m = X_{t[s^m(j-1)+1 : s^m(j-1)+p^m]}, (j = 1, 2, \dots, J^m) \tag{2}$$

where $\hat{X}_t^m$ represents the output of the $p^m - patching$ module, and $\hat{X}_{t,j}^m$ is its jth column.

4.3. Fourier graph spectral network

FGSN can uniformly capture spatiotemporal dynamics through the structure of the hyperpatch graph, and model the task of spatiotemporal time series forecasting from a purely graphical perspective.

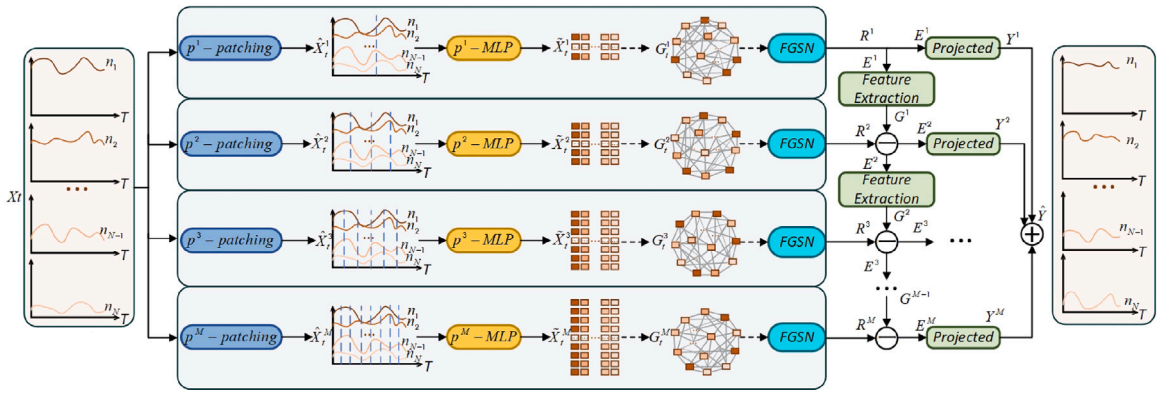


Fig. 1. Framework diagram of the MPFGSN model. The dashed lines represent potential hyperpatch graph relationships that need to be learned through subsequent FGSN.

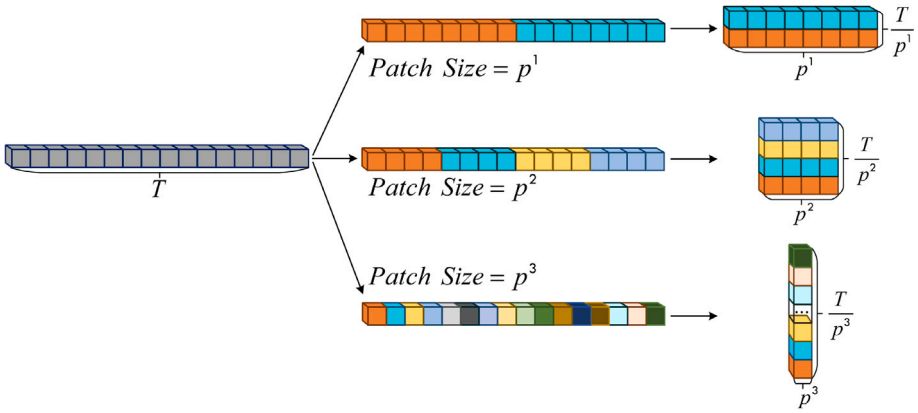


Fig. 2. Schematic diagram of Patching.

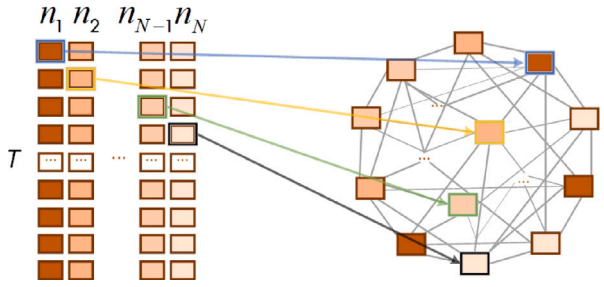


Fig. 3. Schematic diagram of Hyperpatch.

4.3.1. Hyperpatch graph

At the mth layer of the structure, the patches $\hat{X}_t^m$ processed by the $p^m - patching$ are transformed by a multi-layer perceptron network known as $p^m - MLP$ into $\tilde{X}_t^m \in \mathbb{R}^{J^m \times N}$, which contains the feature information of the patches. The role of this $p^m - MLP$ network is to extract features within the patches and perform dimensionality reduction.

The structure of the hyperpatch graph is shown in Fig. 3. In this structure, the feature of each patch, denoted as $\tilde{X}_t^m \in \mathbb{R}^{J^m \times N}$, is defined as a node in the graph, and the sliding window of the time series is represented in the form of a spatiotemporal fully connected graph. Based on $\tilde{X}_t^m \in \mathbb{R}^{J^m \times N}$, a hyperpatch graph structure $G_t^m = (X_t^{G,m}, A_t^{G,m})$ is further constructed, which is initialized as a fully connected graph containing $J^m \times N$ nodes. Here, $X_t^{G,m} \in \mathbb{R}^{(J^m \times N) \times 1}$ represents the node features, and $A_t^{G,m} \in \mathbb{R}^{(J^m \times N) \times (J^m \times N)}$ is the adjacency matrix that describes the relationships between nodes.

The hypergraph structure is specifically designed to address the complex spatiotemporal relationships in spatiotemporal time series forecasting. Integrating information across various time scales, enhances the model's capability to capture the dynamics of time series, thereby improving predictive accuracy and computational efficiency.

4.3.2. Fourier-based spectral convolution

In this section, we will explore the key concepts and theoretical foundation of the FGSN, a novel graph convolution method that leverages the DFT to facilitate graph spectral transformation, simplifying computational procedures and boosting efficiency.

Theorem 1 ([41]). *For two signals x and y on graph G, their graph convolution $x *_G y$ can be represented by the Graph Spectral Transform (GST) $\mathcal{F}_G$ as:*

$$x *_G y = \mathcal{F}_G^{-1}(\mathcal{F}_G(x) \cdot \mathcal{F}_G(y)) \tag{3}$$

where $\mathcal{F}_G$ denotes the GST and $\mathcal{F}_G^{-1}$ denotes its inverse transform.

Since the GST relies on a pre-defined graph structure and involves eigenvalue solving which is usually computationally expensive, we can use DFT to indirectly achieve GST.

Theorem 2. *There exists a graph spectral transformation matrix P such that*

$$\mathcal{F}_G(X) = P^T \mathcal{F}(X) \tag{4}$$

where $\mathcal{F}(X)$ represents the DFT, and $\mathcal{F}_G(X)$ represents the graph spectral transformation. The matrix P, serving as the graph spectral transformation matrix, must satisfy the orthogonality condition $P^T P = I$, the symmetry condition $P = P^T$, and the invertibility condition $PP^{-1} = I$, ensuring that it is a linear transformation.

Proof. Since the DFT and the GST are essentially coordinate transformations under two different orthogonal bases [42], for the DFT $\mathcal{F}(X) = F^T X$ and the GST $\mathcal{F}_G(X) = U^T X$, there exists a GST matrix P such that $U = FP$, where the matrix P satisfies the orthogonality of $P^T P = I$, and the symmetry of $P = P^T$. And the reversibility of $PP^{-1} = I$, and ensure that it is a linear transformation.

Therefore, the GST can also be expressed as:

$$\mathcal{F}_G(X) = U^T X = P^T F^T X = P^T \mathcal{F}(X) \quad \square \tag{5}$$

Given a graph $G_t^m = (X_t^{G,m}, A_t^{G,m})$, according to Theorem 2, the GST can be obtained as follows:

$$\mathcal{F}_G(X_t^{G,m}) = P^T \mathcal{F}(X_t^{G,m}) \tag{6}$$

Therefore, the implementation of the GST is assumed to be done according to Eq. (6). In practical applications, to simplify the model and computational process, the constraints in Theorem 2 can be appropriately relaxed or simplified in exchange for improved computational efficiency.

Definition 1. Introduce a convolutional kernel κ, let $\mathcal{S}_\kappa := \mathcal{F}_G(\kappa) \in \mathbb{C}^{(J^m \times N)\times 1\times 1}$, and define Spectral *Convolution* (*SC*) as $X \cdot \mathcal{S}_\kappa = X \cdot \mathcal{F}_G(\kappa)$, where $\mathcal{S}_\kappa$ denotes the operator of *SC*, and X represents the input matrix, with $\mathcal{F}_G$ indicating the GST.

According to Theorem 1, we have:

$$\mathcal{F}_G(X_t^{G,m} *_G \kappa) = \mathcal{F}_G(X_t^{G,m}) \cdot \mathcal{F}_G(\kappa) = \mathcal{F}_G(X_t^{G,m}) \cdot \mathcal{S}_\kappa \tag{7}$$

It can be seen from the above equation that performing dot product operation on $\mathcal{F}_G(X_t^{G,m})$ and $\mathcal{S}_\kappa$ in the graph spectral domain is equivalent to performing graph convolution operations in the spatiotemporal domain [37].

4.3.3. FGSN

In this section, we will introduce several key mathematical lemmas and theorems that provide a solid foundation for understanding the theoretical basis of FGSN.

Before delving into FGSN, we first need to understand the relationship between graph convolution and SC. Lemma 1 elaborates on the relationship between them.

Lemma 1. *Given the input matrix X, convolutional kernels κ_i ($i = 1, 2, \ldots, n$), operators $\mathcal{S}_j$ of SC ($j = 1, 2, \ldots, n$), and $\mathcal{F}_G$ representing the GST, there exists the following equation between κ_i and $\mathcal{S}_j$:*

$$\mathcal{F}_G(X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i) = \mathcal{F}_G(X) \cdot \prod_{j=1}^{i} \mathcal{S}_j \tag{8}$$

The proofs of Lemma 1 are provided in the Appendix A.

Continuing our discussion, Lemma 2 elaborates on an important property of the GST, namely, that the transform of a sum is equivalent to the sum of the transforms.

Lemma 2. *Given input matrices L_i ($i = 1, 2, \ldots, n$), and $\mathcal{F}_G$ representing the GST, the following formula is established [43]:*

$$\mathcal{F}_G\left(\sum_{i=1}^{n} L_i\right) = \sum_{i=1}^{n} \mathcal{F}_G(L_i) \tag{9}$$

Having established the aforementioned lemmas, we now turn to Theorem 3.

Theorem 3. *Given the input matrix X, convolutional kernels $\kappa_i (i = 1, 2, \ldots, n)$, operators $\mathcal{S}_j$ of SC ($j = 1, 2, \ldots, i$), and $\mathcal{F}_G$ representing the GST, the pointwise multiplication in the graph spectral domain followed by summation with $\mathcal{S}_j$ is equivalent to multi-layer graph convolutions in the spatiotemporal domain:*

$$\mathcal{F}_G\left(\sum_{i=1}^{n} X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i\right) = \sum_{i=1}^{n} \mathcal{F}_G(X) \cdot \prod_{j=1}^{i} \mathcal{S}_j \tag{10}$$

Proof. Based on Lemmas 1 and 2 we can draw the following conclusions:

$$\begin{aligned} &\mathcal{F}_G\left(\sum_{i=1}^{n} X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i\right) \\ &= \sum_{i=1}^{n} \mathcal{F}_G(X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i) \\ &= \sum_{i=1}^{n} \mathcal{F}_G(X) \cdot \prod_{j=1}^{i} \mathcal{S}_j \quad \square \end{aligned} \tag{11}$$

According to Theorem 3, it has been proven that the pointwise multiplication and summation of FGSN in the graph spectral domain are equivalent to multi-layer graph convolutions in the spatiotemporal domain.

The framework diagram of the multi-layer graph convolutions and FGSN is shown in Fig. 4. Specifically, given a node feature $X_t^{G,m} \in \mathbb{R}^{(J^m \times N)\times 1}$ and the adjacency matrix $A_t^{G,m} \in \mathbb{R}^{(J^m \times N)\times (J^m \times N)}$ of the graph $G_t^m = (X_t^{G,m}, A_t^{G,m})$, the computation of the K-layer Fourier Graph Spectral Network can be defined as follows:

$$\begin{aligned} R^m &= FGSN(X_t^{G,m}, A_t^{G,m}) \\ &= \sum_{k=1}^{K} \sigma(\mathcal{F}_G(X_t^{G,m}) \mathcal{S}_{0:k} + b_k), \quad \mathcal{S}_{0:k} = \prod_{i=1}^{k} \mathcal{S}_i \end{aligned} \tag{12}$$

where $\mathcal{S}_i$ is the *SC* operator of the ith layer, satisfying $\mathcal{F}_G(X_t^{G,m})\mathcal{S}_i = \mathcal{F}_G(X_t^{G,m} *_G \kappa)$, $\mathcal{S}_0$ is the identity matrix, and $b_k \in \mathbb{C}^1$ is the complex-valued bias parameter. All operations in the FGSN are performed in the graph spectral domain. Therefore, all parameters, i.e., $\{\mathcal{S}_k, b_k\}_{k=1}^{K}$, are complex numbers.

According to Theorem 3, the point-wise multiplication and summation of *SC*s in the graph spectral domain (Fig. 4 bottom) are equivalent to the multi-layer graph convolutions in the spatiotemporal domain (Fig. 4 top) :

$$\begin{aligned} &\mathcal{F}_G^{-1}\left(\sum_{i=1}^{n} \mathcal{F}_G(X_t^{G,m}) \cdot \mathcal{S}_{0:k_i}\right) \\ &= \sum_{i=1}^{n} X_t^{G,m} *_G \kappa_1 *_G \kappa_2 \cdots *_G \kappa_i, \quad \mathcal{S}_{0:k_i} = \prod_{j=0}^{k_i} \mathcal{S}_j \end{aligned} \tag{13}$$

where $\mathcal{F}_G{}^{-1}$ represents the inverse GST.

4.3.4. Complexity analysis of FGSN

In FGSN, the graph spectral transformation is implemented through the spectral transformation matrix $P \in \mathbb{R}^{q\times n}$, where n is the total number of nodes in the graph, and q is the number of bases in the spectral domain space. Since the information in the spectral domain space is mainly concentrated on the bases with larger eigenvalues, it is possible to set q much less than n ($q \ll n$), which helps to reduce computational complexity.

The time complexity of *SC* encompasses the operations of Fast Fourier Transform (FFT), Inverse Fast Fourier Transform (IFFT), and matrix multiplication in the spectral domain space, with the time complexities of FFT and IFFT being $O(n \log n)$. Consequently, the time complexity for a single layer of *SC* is $O(nd \log n + qnd + qd)$, and the total time complexity for the FGSN is $O(nd \log n + qnd + Kqd)$, where K denotes the number of *SC* layers, and d denotes the number of features. Due to the spectral method's time complexity of $O(n \log n)$, it is usually more efficient than the spatiotemporal domain multi-layer graph convolutions operation, which has a time complexity of $O(n^2 d + nd^2)$.

The FGSN model demonstrates exceptional performance in the field of time series prediction, particularly in terms of computational efficiency, which is significantly superior to other graph-based methods. This high efficiency is mainly attributed to its innovative spectral convolution technique, which effectively reduces the complexity of

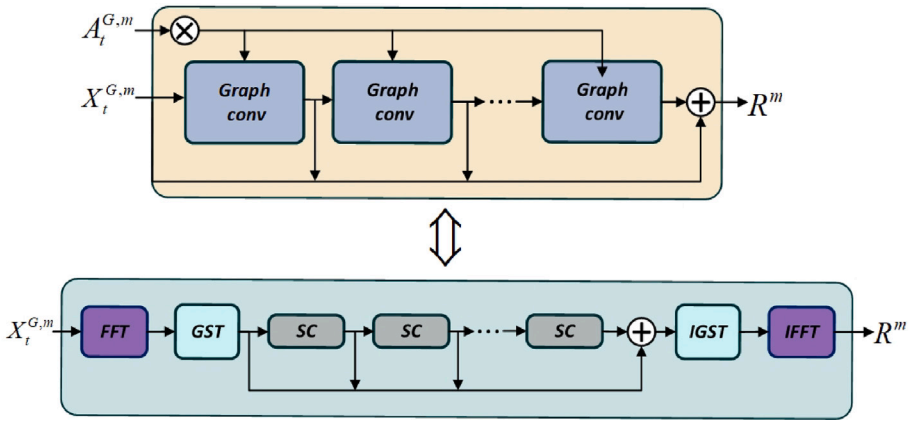


Fig. 4. Schematic diagrams of multi-layer graph convolutions (top) and FGSN (bottom) frameworks.

spectral computation and successfully avoids the eigenvalue decomposition issues common in methods such as SCNN and ChebNet. Although the time complexity of FGSN is $O(nd \log n + qnd + Kqd)$, which is asymptotically $O(n \log n)$ like FourierGNN's $O(nd \log n + Knd^2)$, when dealing with spatiotemporal datasets with a large number of feature variables, FourierGNN's time complexity exceeds that of FGSN. While FGSN exhibits lower time complexity compared to FourierGNN when dealing with spatiotemporal datasets with a large number of feature variables, the introduction of multi-granularity in FGSN improves performance but may lead to an increase in time complexity, potentially exceeding that of FourierGNN. Therefore, MPFGSN also further reduces memory burden by using patches.

4.4. Multi-resolution residual fusion

In time series forecasting, multi-resolution data provides insights at different time scales, but it also poses challenges of information redundancy and semantic overlap, which can affect model performance. Inspired by the work in [17], we address these issues and enhance the accuracy of predictions using a multi-resolution residual fusion approach.

4.4.1. Residual learning

To effectively harness information across various resolutions, we employ a block learning approach where each module focuses on a specific resolution. Low-resolution data typically represents a coarse approximation of the information, containing fewer details and lower sampling frequency. Although it lacks the fine-grained details of high-resolution data, it still contains useful structural or contextual information that can guide the learning process at higher resolutions. This prior information from low-resolution data helps to form a foundational understanding of the overall patterns or structures, which can be refined when processing higher-resolution data. High-resolution data, on the other hand, captures more detailed information, typically obtained through higher sampling frequencies or finer granularity. While high-resolution data provides a richer representation, it can also be more computationally expensive to process.

To address redundancy, which is the overlapping or repeated information between low-resolution and high-resolution data, we combine modules in a low-to-high cascade. Each module is designed to process only its corresponding resolution, with de-redundancy operations ensuring that redundant information from lower resolutions is removed when processing higher resolutions. This allows each module to focus on the unique, non-overlapping knowledge at its specific resolution. Specifically, the process involves first extracting features G^{m-1} at resolution $m-1$, then subtracting the redundant information at resolution m. This enables the module at resolution m to concentrate on learning the unique aspects of that resolution.

We formulate the cross-resolution residual learning process as:

$$E^m = \begin{cases} R^m, & \text{if } m = 1, \\ R^m - G^{m-1}, & \text{otherwise.} \end{cases} \tag{14}$$

where E^m represents the de-redundant input at resolution m.

To obtain the unique features learned at each layer of resolution, we use a linear layer projection to obtain the learning features G^m at a specific resolution:

$$G^m = F_{\mathrm{Re}\,c}^{Linear}(E^m) \tag{15}$$

4.4.2. Similarity weighting

To obtain the prediction results at each layer, we employ a two-layer linear structure to derive the specific resolution predictions Y^m:

$$Y^m = F_{\mathrm{Pr}\,ed}^{Linear}(E^m) \tag{16}$$

We then weight and sum the predictions $Y^1, Y^2, \ldots, Y^M$ based on similarity. It is presumed that each Y^m and the true label y adhere to a normal distribution. Within the training process, the label y is utilized to compute the Pearson correlation coefficient between Y^m and y, serving as a metric for similarity, as illustrated in the subsequent formula:

$$\gamma^m = \frac{\sum_{i=1}^{n} (Y_i^m - \bar{Y}_i^m)(y_i - \bar{y}_i)}{\sqrt{\sum_{i=1}^{n} (Y_i^m - \bar{Y}_i^m)^2} \sqrt{\sum_{i=1}^{n} (y_i - \bar{y}_i)^2}} \tag{17}$$

where γ^m represents the similarity between Y^m and the label y, n denotes the number of sample points in Y^m and y, $\bar{Y}^m$ is the mean of the mth layer's prediction results, and $\bar{y}_i$ is the mean of the label y.

We then use the Softmax function to convert these similarities into weight coefficients:

$$\omega^m = \frac{e^{\gamma^m}}{\sum_{i=1}^{M} e^{\gamma^i}} \tag{18}$$

Finally, we multiply each layer's prediction result by the corresponding weight coefficient and sum the weighted results from all layers to obtain the final prediction result:

$$\hat{Y}_{train} = \sum_{i=1}^{M} Y^m \omega^m \tag{19}$$

All the above processes are carried out in the training set. In the test set, we use the average weight of each layer retained from the training set to obtain the final prediction result:

$$\hat{Y}_{test} = \sum_{i=1}^{M} Y^m \bar{\omega}^m \tag{20}$$

where $\bar{\omega}^m$ is the average weight of the mth layer obtained from the training set.

Table 1
Dataset statistics.

Datasets	ETTm1	ETTm2	ETTm2	ETTh2	ECG	exchange_rate	PeMS07	CSI300	METR-LA
Features	7	7	7	7	140	8	228	100	207
Timesteps	69 680	69 680	17 420	17 420	5000	7588	12 672	2791	34 272

Algorithm 1 Computation Process of MPFGSN

Input: $X_t \in \mathbb{R}^{T\times N}$
Output: $\hat{Y} \in \mathbb{R}^{\mathcal{T}_{Pred}\times N}$

1: Perform p^m-**Patching** on X_t according to Equation (2) to obtain $\hat{X}_t^m$;
2: Feed X_t^m into p^m-**MLP** to get $\tilde{X}_t^m$;
3: Construct **Hyperpatch Graph** $G_t^m = (X_t^{G,m}, A_t^{G,m})$ from $\tilde{X}_t^m$;
4: Input $X_t^{G,m}$ into **FGSN** according to Equation (12) to obtain R^m;
5: Conduct **Residual Learning** on R^m according to Equation (14) to get E^m;
6: Perform **Similarity weighting** on E^m according to Equations (16) - (20) to obtain $\hat{Y}$;
7: **return** $\hat{Y}$;

4.5. Loss function and training of MPFGSN

4.5.1. Computation process of MPFGSN

The forward computation of MPFGSN is the process by which the model obtains the prediction results from the input data. It goes through several steps including multi-resolution patch processing, multi-layer perceptron transformation, hyperpatch graph construction, Fourier graph spectral network computation, residual learning, and similarity weighting. The algorithm for the forward computation of MPFGSN is shown in Algorithm 1.

4.5.2. Loss function

MPFGSN uses the Mean Squared Error (MSE) loss function to measure the difference between the predicted value $\hat{Y}$ and the true value Y. The expression of the loss function is as follows:

$$Loss = MSE = \frac{1}{\mathcal{T}_{\Pr ed} N} \sum_{i=1}^{\mathcal{T}_{\Pr ed}} \sum_{j=1}^{N} \left(Y_{train}^{i,j} - \hat{Y}_{train}^{i,j} \right)^2 \tag{21}$$

where $\mathcal{T}_{Pred}$ is the number of prediction time steps, N is the data dimension, $Y_{train}^{i,j}$ represents the true value from the training set at the ith time step and jth dimension, and $\hat{Y}_{train}^{i,j}$ is the corresponding predicted value for the training set at the same time step and dimension.

4.5.3. Back-propagation

During MPFGSN training, RMSprop is used for back-propagation. In the forward pass, input X_t to obtain $\hat{Y}$ according to Algorithm 1, and then compute the MSE loss Loss between $\hat{Y}$ and the true label Y. In back-propagation, calculate the gradients of trainable parameters concerning Loss using the chain rule, and RMSprop updates the parameters based on these gradients and an adaptive learning rate. FFT transforms data in the forward pass while IFFT aids in gradient computation in the backward pass. The training process repeats these steps until the model converges or the preset number of epochs is reached.

5. Experiments

5.1. Datasets and settings

5.1.1. Datasets

To evaluate the performance of MPFGSN, we conducted experiments on nine public benchmark datasets: ETTm1, ETTm2, ETTh1, ETTh2, ECG, exchange_rate, PeMS07, CSI300, and METR-LA. Table 1 provides a summary of the dataset statistics, and the details of the nine public benchmark datasets are as follows:

- ETTm1, ETTm2, ETTh1, ETTh2[1]: The ETT dataset is a series of measurements of electrical transformers provided by Zhou et al. [6] from 2016 to 2018, including load and oil temperature data.
- ECG[2]: This dataset pertains to electrocardiograms (ECG) and is from the UCR Time Series Classification Archive [44]. It contains 140 nodes, each with a length of 5000.
- exchange_rate[3]: This dataset records the daily exchange rates of eight different countries from 1990 to 2016 [45].
- PeMS07[4]: This dataset consists of traffic flow data from California highways, collected in real-time every 30 s by the California Transportation Performance and Measurement System (PeMS) [46]. The original traffic flow data is aggregated at 5-min intervals. The dataset records the geographical information of sensor stations. PeMS07 is a dataset from one of the regions.
- CSI300[5]: This is a stock dataset, compiling the closing prices of one hundred stocks that have been continuously listed in the CSI 300 from 2012 to 2024.
- METR-LA[6]: The dataset includes traffic information collected by loop detectors on highways in Los Angeles County from March 1, 2012, to June 30, 2012. It contains data from 207 sensors, with samples taken every 5 min.

We divided the complete ETT dataset into a training set (60%), validation set (20%), and test set (20%) in chronological order. The other datasets were similarly divided into a training set (70%), validation set (10%), and test set (20%) in chronological order.

5.1.2. Experimental settings

The MPFGSN model is implemented in Python using PyTorch 2.2.0 and is trained on a GPU (NVIDIA GeForce RTX 4090), and source codes are available on github.[7] We set the input window size T to 96 or 192 and the learning rate to 0.0001. The RMSprop optimizer is used to optimize all trainable parameters through backpropagation. All of the models follow the same experimental setup with prediction window sizes $H \in \{96, 192, 336, 720\}$ for the ETTm1, ETTm2, ETTh1, ETTh2, PeMS07, and METR-LA datasets, $H \in \{48, 96, 192, 336\}$ for the ECG and exchange_rate datasets, and $H \in \{36, 48, 60, 96\}$ for the CSI300 dataset. The longer the prediction window, the greater the difficulty of forecasting. For more experimental results, please refer to Appendix B.

5.1.3. Evaluation metrics

We employ Mean Absolute Error (MAE) and Mean Squared Error (MSE) as evaluation metrics, which are defined as follows:

$$MAE = \frac{1}{\mathcal{T}_{\Pr ed} N} \sum_{i=1}^{\mathcal{T}_{\Pr ed}} \sum_{j=1}^{N} \left| Y_{test}^{i,j} - \hat{Y}_{test}^{i,j} \right| \tag{22}$$

[1] The ETT dataset is publicly available at https://github.com/thuml/Autoformer.
[2] The ECG dataset is publicly available at https://www.timeseriesclassification.com/description.php?Dataset=ECG5000.
[3] The exchange_rate dataset is publicly available at https://github.com/thuml/Autoformer.
[4] The PeMS07 dataset is publicly available at https://github.com/microsoft/StemGNN.
[5] The CSI300 dataset is publicly available at https://tushare.pro/.
[6] The METR-LA dataset is publicly available at https://github.com/liyaguang/DCRNN.
[7] https://github.com/komorebi424/MPFGSN.

Table 2
Summary of results for all methods on nine datasets. The best results are in bold and the second best are underlined.

Model		MPFGSN		CycleNet		MiTSformer		iTransformer		PatchTST		FourierGNN		TimesNet		DLinear		Crossformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.3635**	0.3985	0.3670	**0.3980**	0.4151	0.4280	0.3909	0.4151	0.3752	0.3996	0.4633	0.4696	0.4403	0.4425	0.3772	0.4009	0.4260	0.4365
	192	**0.4273**	0.4340	0.4382	0.4336	0.4838	0.4649	0.4575	0.4457	0.4453	0.4356	0.5187	0.4994	0.4901	0.4757	0.4431	**0.4333**	0.4616	0.4602
	336	**0.4864**	0.4735	0.5087	0.4686	0.5444	0.5051	0.5326	0.4851	0.5162	0.4736	0.6335	0.5688	0.6198	0.5239	0.5076	**0.4684**	0.5266	0.4997
	720	**0.5483**	0.5145	0.5823	0.5176	0.6054	0.5465	0.6048	0.5295	0.5881	0.5184	0.6943	0.6017	0.6883	0.5673	0.5682	**0.5120**	0.6407	0.5719
ETTm2	96	0.1659	0.2750	**0.1434**	**0.2610**	0.1912	0.3108	0.1682	0.2750	0.1588	0.2640	0.2333	0.3355	0.1625	0.2648	0.1731	0.2848	0.2016	0.3041
	192	0.2018	0.3045	**0.1718**	**0.2847**	0.2388	0.3528	0.2010	0.3006	0.1933	0.2939	0.3657	0.4238	0.2007	0.2969	0.2172	0.3232	0.2834	0.3733
	336	0.2513	0.3439	**0.2078**	**0.3123**	0.2739	0.3724	0.2411	0.3293	0.2354	0.3253	0.5757	0.5409	0.2499	0.3296	0.2722	0.3661	0.3680	0.4297
	720	**0.2875**	**0.3701**	0.2978	0.3773	1.5272	0.7463	0.3145	0.3734	0.3075	0.3708	0.9310	0.6737	0.3386	0.3843	0.3592	0.4253	0.4577	0.4836
ETTh1	96	0.4723	0.4755	**0.4555**	**0.4620**	0.4602	0.4691	0.5835	0.5331	0.4689	0.4638	0.5154	0.5068	0.4689	0.4798	0.5172	0.4971	0.4883	0.4965
	192	**0.5237**	0.5110	0.5342	0.5140	0.5279	0.5218	0.6789	0.5836	0.5263	**0.5017**	0.5703	0.5429	0.5453	0.5266	0.5637	0.5295	0.5558	0.5411
	336	0.5758	0.5478	**0.5592**	**0.5292**	0.5766	0.5451	0.7582	0.6221	0.5732	0.5328	0.6220	0.5782	0.6282	0.5823	0.6043	0.5564	0.8854	0.6945
	720	**0.6790**	**0.6140**	0.7111	0.6180	0.7257	0.6328	0.9313	0.7100	0.7228	0.6221	0.7158	0.6355	0.8416	0.7051	0.7122	0.6221	1.0839	0.7817
ETTh2	96	**0.2301**	0.3379	0.2728	0.3657	0.2757	0.3722	0.2744	0.3574	0.2394	**0.3265**	0.4222	0.4591	0.2572	0.3417	0.2779	0.3702	0.3398	0.4040
	192	**0.2515**	**0.3539**	0.3036	0.3853	0.3648	0.4252	0.3320	0.3923	0.2925	0.3608	0.4966	0.5277	0.3139	0.3799	0.3355	0.4106	0.9537	0.7486
	336	**0.2795**	**0.3815**	0.3256	0.3993	0.3921	0.4522	0.3741	0.4169	0.3348	0.3876	0.6164	0.5931	0.3464	0.4006	0.3761	0.4423	1.1541	0.8319
	720	**0.3729**	0.4395	0.3830	**0.4338**	0.4447	0.4819	0.4573	0.4629	0.4284	0.4431	0.8576	0.7242	0.4134	0.4422	0.4577	0.4943	1.1153	0.8260
ECG	48	**1.0491**	**0.7320**	1.0946	0.7525	1.0675	0.7377	1.1330	0.7683	1.1815	0.7861	1.1287	0.7720	1.0840	0.7477	1.0751	0.7494	1.3132	0.8615
	96	**1.1343**	**0.7691**	1.1804	0.7900	1.1710	0.7815	1.3810	0.8637	1.2682	0.8228	1.2036	0.8053	1.1669	0.7845	1.1520	0.7833	1.3769	0.8875
	192	1.3116	0.8524	1.3467	0.8612	1.3823	0.8717	1.3602	0.8662	1.4302	0.8903	1.3336	0.8600	1.3369	0.8577	**1.3100**	**0.8509**	1.5375	0.9516
	336	**1.4105**	**0.9025**	1.5085	0.9253	1.5811	0.9484	1.5088	0.9256	1.5953	0.9534	1.4353	0.9041	1.5400	0.9332	1.4482	0.9079	1.5847	0.9706
exchange_rate	48	0.0530	0.1675	0.0550	0.1671	**0.0474**	**0.1486**	0.0606	0.1757	0.0567	0.1694	0.0563	0.1708	0.0587	0.1726	0.0906	0.2299	0.4343	0.4913
	96	**0.0856**	0.2123	0.0889	**0.2114**	0.1148	0.2291	0.1137	0.2394	0.1005	0.2248	0.1644	0.3019	0.1053	0.2351	0.1310	0.2731	0.5604	0.5737
	192	**0.1730**	0.3100	0.1798	0.3032	0.1857	**0.2957**	0.2091	0.3296	0.1977	0.3184	0.2405	0.7972	0.2379	0.3552	0.2081	0.3458	0.8541	0.7224
	336	**0.3033**	**0.4151**	0.3310	0.4164	0.3921	0.4390	0.3610	0.4384	0.3515	0.4307	1.1836	0.8317	0.3765	0.4540	0.3072	0.4182	1.0893	0.8269
PeMS07	96	0.6418	0.5341	1.2801	0.7449	0.9081	0.5906	1.4234	0.8406	1.3967	0.8154	**0.5894**	**0.4871**	0.9642	0.6299	1.2379	0.8239	0.9710	0.6886
	192	**0.5401**	**0.4640**	1.2955	0.7452	0.8967	0.5772	1.4606	0.8505	1.4406	0.8258	0.5881	0.4972	1.0033	0.6422	1.2341	0.8220	1.0824	0.7412
	336	**0.5334**	**0.4624**	1.0228	0.6279	0.7593	0.5193	1.2137	0.7500	1.1388	0.6970	0.5595	0.4801	0.8747	0.5883	1.0336	0.7224	1.0983	0.7614
	720	**0.5515**	**0.4710**	1.1140	0.6662	0.8036	0.5336	1.3193	0.7908	1.2398	0.7364	0.5681	0.4863	1.0827	0.6804	1.0911	0.7464	1.0584	0.7482
CSI300	36	**0.2480**	**0.2729**	0.2613	0.3144	0.2574	0.2970	0.2906	0.3310	0.3024	0.3413	0.3860	0.3262	0.3286	0.3514	0.3351	0.3725	3.7459	0.9251
	48	0.3578	**0.3191**	**0.2995**	0.3372	0.3401	0.3411	0.3375	0.3593	0.3389	0.3631	0.3944	0.3541	0.3597	0.3688	0.3617	0.3865	3.8492	0.9590
	60	0.5308	0.3758	**0.3361**	**0.3592**	0.5111	0.4111	0.3771	0.3817	0.3770	0.3847	0.7114	0.4253	0.3870	0.3907	0.3870	0.3992	3.9137	0.9745
	96	0.6551	0.4412	**0.4266**	0.4122	0.6084	0.4585	0.4882	0.4455	0.4279	**0.4120**	1.2628	0.5552	0.5322	0.4686	0.4652	0.4471	4.6413	1.1842
METR-LA	96	**1.1074**	0.7300	1.3086	**0.6947**	1.3261	0.7250	1.3427	0.7392	1.3814	0.7379	1.1737	0.7085	1.3643	0.7241	1.1336	0.7360	1.1165	0.7170
	192	**1.2203**	**0.7398**	1.4998	0.7582	1.4698	0.7821	1.5222	0.8002	1.5377	0.7844	1.2967	0.7678	1.5492	0.7796	1.2338	0.7750	1.2713	0.7825
	336	1.3179	**0.7513**	1.5361	0.7568	1.5705	0.7883	1.5608	0.8010	1.5539	0.7790	1.3287	0.7842	1.5148	0.7727	**1.2652**	0.7818	1.2911	0.8094
	720	1.4788	**0.7936**	1.7974	0.8421	1.7972	0.8438	1.9053	0.9102	1.8764	0.8780	1.4944	0.8244	1.8171	0.8532	**1.3824**	0.8165	1.4283	0.8309
Avg rank		**2.1111**	**2.1389**	3.2778	2.7778	5.2222	5.0833	6.1667	6.1944	4.7778	4.3889	6.4167	6.3056	5.3056	5.1667	4.6389	5.1944	7.0833	7.7500

$$MSE = \frac{1}{\mathcal{T}_{\Pr ed} N} \sum_{i=1}^{\mathcal{T}_{\Pr ed}} \sum_{j=1}^{N} \left(Y_{test}^{i,j} - \hat{Y}_{test}^{i,j} \right)^2 \tag{23}$$

where $\mathcal{T}_{Pred}$ is the number of prediction time steps, N is the data dimension, $Y_{test}^{i,j}$ represents the true value from the test set at the ith time step and jth dimension, and $\hat{Y}_{test}^{i,j}$ is the corresponding predicted value for the test set at the same time step and dimension.

5.2. Methods for comparison

We have selected state-of-the-art (SOTA) models based on the Transformer, including PatchTST [22], Crossformer [21], and iTransformer [47], as well as recent non-Transformer-based models such as CycleNet [7], FourierGNN [13], TimesNet [25], and DLinear [48], along with the recent spatiotemporal model MiTSformer [6], as comparative methods for our model. We utilize the code published in the original papers, with experimental settings aligned with our approach, and hyperparameter configurations are based on the original papers.

5.3. Main results

Table 2 presents the experimental results of MPFGSN compared to all comparative methods across nine datasets, leading to the following conclusions:

- The MPFGSN model demonstrates superior performance across various datasets, with average rankings of 2.1111 for MSE and 2.1389 for MAE, indicating that the model has reached an advanced level on these datasets. Particularly, on the ECG and PeMS07 datasets, the MPFGSN model's performance is markedly superior, outperforming other comparative methods in both evaluation metrics and across all prediction window lengths. In contrast, the CycleNet model has an advantage on the ETT dataset, but it does not show a significant advantage on the ECG and PeMS07 datasets. This notable difference may be attributed to the strong spatiotemporal relationships present in both the ECG and traffic datasets, which are highly suitable for analysis and prediction by the MPFGSN model.
- In SOTA Transformer-based models, the PatchTST model has achieved significant results in time series data analysis due to its unique input method—using patches to capture local temporal patterns. However, as the number of variables increases, the MPFGSN model has demonstrated performance that surpasses the PatchTST model on datasets with a large number of variables. This phenomenon may be attributed to the fact that as the number of variables grows, the interactions between variables in the data become more critical. The PatchTST model primarily focuses on the independence of channels, which limits its ability to capture features of variable interaction. In contrast, the MPFGSN model effectively captures these interactive features, giving it an advantage in handling more complex time series data. As the number of variables increases, this capability becomes especially critical, as it allows the model to perform more comprehensive and detailed feature learning. This in-depth learning of variables enables the MPFGSN model to provide higher predictive accuracy when dealing with predictions involving more variables. Therefore, when dealing with time series data that involves interactions among multiple variables, the MPFGSN model becomes a more reliable choice due to its advantage in capturing variables.
- The FourierGNN model performs well on the exchange_rate, PeMS07, and CSI300 datasets, attributed to the significant spatiotemporal dependencies in these datasets. However, as the prediction window lengthens, the FourierGNN model does not maintain its advantage, as long sequence predictions require the model to capture more detailed features to maintain accuracy. In comparison, the MPFGSN model effectively learns and processes the internal features of the data through multi-resolution patches and a residual fusion mechanism, significantly enhancing its ability to handle details. This multi-resolution approach allows our model to perform well across all prediction window lengths, especially in long-term forecasting tasks, where the precision of the MPFGSN is notably improved.

Table 3
Ablation study results: MPFGSN model variants and their performance on PeMS07, CSI300, and exchange_rate datasets. The best results are in bold and the second best are underlined.

Methods		PeMS07				CSI300				exchange_rate			
		96	192	336	720	36	48	60	96	48	96	192	336
MPFGSN	MSE	**0.6418**	**0.5401**	**0.5334**	**0.5515**	**0.2480**	**0.3578**	**0.5308**	**0.6551**	**0.0530**	**0.0856**	**0.1730**	**0.3033**
	MAE	**0.5341**	**0.4640**	**0.4624**	**0.4710**	**0.2729**	**0.3191**	**0.3758**	**0.4412**	**0.1675**	**0.2123**	**0.3100**	**0.4151**
Rep-Fine2Coarse	MSE	0.7636	0.7755	0.6923	0.7142	0.2791	0.3778	0.7988	0.8605	0.0619	0.1068	0.1944	0.5484
	MAE	0.5912	0.5976	0.5425	0.5639	0.3007	0.3384	0.5077	0.4774	0.1845	0.2420	0.3299	0.5631
Rep-RandOrder	MSE	0.7394	0.7544	0.6808	0.6987	0.3647	0.4229	0.6295	0.8620	0.0580	0.1401	0.1976	0.7433
	MAE	0.5763	0.5849	0.5313	0.5527	0.3457	0.3472	0.4408	0.4774	0.1747	0.2803	0.3307	0.6436
w/o-Patching	MSE	0.7468	0.7624	0.6801	0.7049	0.2892	0.6512	0.5493	0.6843	0.0762	0.1244	0.3486	0.4776
	MAE	0.5756	0.5809	0.5422	0.5532	0.3232	0.5115	0.4154	0.4664	0.2024	0.2653	0.4306	0.5137
w/o-InteractRes	MSE	0.6629	0.5790	0.6872	0.7075	0.2777	0.4261	0.6892	0.6903	0.0583	0.1069	0.2475	0.5779
	MAE	0.5369	0.4986	0.5401	0.5606	0.3167	0.3568	0.4326	0.4753	0.1780	0.2447	0.3654	0.5560
Rep-FeatAdd	MSE	0.7813	0.7558	0.6865	0.7031	0.3106	0.4086	0.6851	0.6823	0.0632	0.1025	0.2704	0.5894
	MAE	0.5838	0.5872	0.5396	0.5582	0.3237	0.3572	0.4293	0.4710	0.1852	0.2367	0.3838	0.5640
Rep-AvgWeight	MSE	0.8984	0.9031	0.8413	0.8616	0.4647	0.7739	1.0391	0.9767	0.2392	0.6734	0.6547	1.4998
	MAE	0.6875	0.6932	0.6333	0.6493	0.3974	0.4804	0.5357	0.5617	0.3830	0.6412	0.6188	0.9355
Rep-FGOs	MSE	0.7519	0.7556	0.6772	0.6904	0.2711	0.5424	0.5648	0.7910	0.0566	0.1067	0.1841	0.4963
	MAE	0.5612	0.5887	0.5318	0.5462	0.2998	0.3913	0.4055	0.4699	0.1768	0.2441	0.3227	0.5209

5.4. Ablation studies

To comprehensively explore the impact of each component and setting on the performance of the MPFGSN model, we designed several variant models. We conducted experiments on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset. The results are summarized in Table 3.

5.4.1. Introduction of variant models

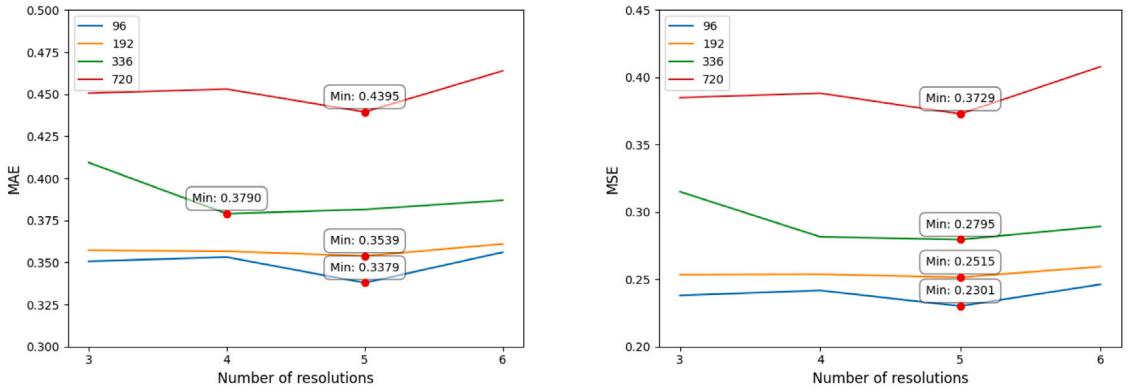
- **Rep - Fine2Coarse**: This variant model has the resolution order reversed from the default setting of the original model, adopting a sorting method from fine to coarse resolutions. This approach attempts to observe the impact of resolution order changes on model performance by first processing more detailed data and then gradually transitioning to more generalized data.
- **Rep - RandOrder**: In this variant, the resolution order is completely randomized, with fine and coarse resolution data interleaved. This design aims to test the performance of the model when faced with an irregular resolution order.
- **w/o - Patching**: To gain a deeper understanding of the role and effectiveness of the Patching method in the MPFGSN model, we designed this variant model for an ablation study, which removes the Patching module and directly conducts dimensionality reduction and feature learning through an MLP.
- **w/o - InteractRes**: In this variant model, the redundant prior signal part subtracted from resolution m to resolution $m-1$ is removed, setting $E^m = R^m$. This design implies that the model does not consider the interaction of information between different resolutions and processes the data at each resolution independently.
- **Rep - FeatAdd**: Unlike the lack variant, the added variant uses feature addition for feature fusion, setting

$$E^m = \begin{cases} R^m, & \text{if } m = 1, \\ R^m + G^{m-1}, & \text{otherwise.} \end{cases} \tag{24}$$

- **Rep - AvgWeight**: To explore the importance of the similarity weighting mechanism in the MPFGSN model, we designed this special variant model. In this variant, the weighting method for the prediction results of each layer was changed: instead of weighting based on the similarity of the prediction results, a simple average weighting method was used.
- **Rep - FGOs**: To verify the role and contribution of the FGSN module in the MPFGSN model, we designed this specific variant model. In this variant, we replaced the FGSN module with the stacking of the FGO module from the FourierGNN paper.

5.4.2. Discussion of experimental results

1. **Effect of resolution size ordering**: The experimental results, as shown in Table 3, indicate that on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset, the original MPFGSN model demonstrated the best performance under all tested conditions. This finding suggests that the original model's method of ordering from coarse to fine resolutions—using coarse resolution data to reconstruct fine resolution data to simulate redundant information—is effective. Our method can effectively address the redundancy issue between different resolution data, preventing the model from being dominated by redundant coarse resolution trend information. Comparative studies show resolution ordering greatly affects MPFGSN model efficacy. Our model optimizes performance by minimizing low - resolution data redundancy. This highlights the importance of considering resolution ordering when designing multi-resolution time series forecasting models.
2. **Effect of Patching**: By comparing the performance of the w/o - Patching variant and the original MPFGSN model on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset, the results shown in Table 3 indicate that the original MPFGSN model significantly outperforms w/o - Patching. Patching technology plays an essential role in the MPFGSN model by dividing time series data into multiple small, continuous time segments (i.e., patches), allowing the model to learn the local features of time series more meticulously. These local features are crucial for understanding the dynamics of time series and predicting future data trends. Therefore, with the removal of the Patching module, the w/o - Patching variant's ability to capture these critical local temporal patterns is limited, leading to a decrease in performance. In summary, Patching technology is an indispensable part of the MPFGSN model, significantly enhancing the model's understanding of time series data by capturing local temporal patterns, thereby improving the accuracy of predictions. This finding emphasizes the importance of considering local feature learning when designing and optimizing time series forecasting models.



(a) MAE (b) MSE

Fig. 5. Results of MPFGSN at different resolutions. The best results are marked with red circles.

3. **Effects of cross-resolution residual learning**: As shown in Table 3, the original MPFGSN model outperformed the w/o - InteractRes and Rep - FeatAdd variants in all cases on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset. The results indicate that the design of the cross-resolution residual learning network is crucial for improving the performance of time series forecasting. The MPFGSN model enhances predictive accuracy by capturing specific resolution details and interactions between resolutions with its cross-resolution residual learning network, and its superior performance over variants confirms the effectiveness of integrating data across different resolutions.
4. **Effect of similarity weighting**: By comparing the performance of the original MPFGSN model and the Rep - AvgWeight variant on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset, the experimental results in Table 3 show that the original MPFGSN model outperforms Rep - AvgWeight in all evaluation metrics. This result emphasizes the role and contribution of the similarity weighting mechanism in the model. The MPFGSN model's similarity weighting mechanism prioritizes more accurate predictions, enhancing overall forecasting by integrating layered information. In contrast, equal weighting may overlook key insights and obtain lower accuracy. This result highlights the crucial role of similarity-based weighting in optimizing time series forecasts.
5. **Effect of FGSN**: The experimental results, as shown in Table 3, indicate that the original MPFGSN model outperformed Rep - FGOs on the PeMS07 dataset, CSI300 dataset, and exchange_rate dataset. This result demonstrates that the FGSN module not only reduces computational complexity but also enhances the accuracy of predictions.

In summary, each component and setting of the MPFGSN model plays an important role in improving its performance. The comparison between each variant model and the original model clarifies the value and contribution of each part in the model.

5.5. Parameter study

To thoroughly investigate the impact of multi-resolution modeling on the performance of the MPFGSN, we evaluated the model with varying numbers of resolutions on the ETTh2 dataset. The numbers of resolutions were 6, 5, 4, and 3, and through these different settings, we observed the performance changes of the MPFGSN under different resolutions.

The experimental results, as shown in Fig. 5, indicate that as the resolution increases, the predictive performance of the MPFGSN model gradually improves. Under different settings of the prediction window length, when the resolution reaches 5, the model has achieved optimal performance in almost all evaluation metrics. This trend suggests that increasing the resolution helps the model to capture more detailed feature information, thereby enhancing the accuracy of the prediction.

However, when the resolution is further increased to 6, the performance of the MPFGSN model does not continue to improve and instead shows a trend of stagnation or even decline. The reason is that the excessive number of resolutions leads to a more complicated learning process for the model. It not only escalates the model's complexity but also poses the risk of overfitting. Such overfitting can impair the model's ability for generalization, and undermine its predictive performance on new, unseen data. Therefore, the setting of resolution at 5 provides a balance for the MPFGSN model, and it can fully utilize the model's learning capabilities while avoiding the risk of overfitting.

5.6. Time consumption analysis

As shown in Table 4, in comparative experiments on the ETTm1, ECG, exchange_rate, and CSI300 datasets, we compared the MPFGSN(5) and MPFGSN(3) models with resolutions of 5 and 3 respectively, along with FourierGNN, MiTSformer, CycleNet, and PatchTST. Regarding time consumption, MPFGSN(3) consumes far less time than the spatiotemporal model MiTSformer and also has lower time consumption than non-spatiotemporal models like FourierGNN, CycleNet, and PatchTST. In terms of effectiveness, MPFGSN(5) significantly outperforms CycleNet and is superior to other models, with remarkable capabilities in capturing complex spatiotemporal patterns. Although MPFGSN(5) has a higher time cost compared to MPFGSN(3), its time overhead is still significantly lower than that of the spatiotemporal model MiTSformer. In conclusion, these experiments prove that our models exhibit good computational efficiency and we can enhance effectiveness by sacrificing some time.

Our results indicate that MPFGSN(3) has better time efficiency than CycleNet. At the same time, MPFGSN(5) achieves higher accuracy at the cost of increased time consumption. This trade-off allows us to choose between time efficiency and model performance depending on the specific requirements of the task. These results collectively indicate that MPFGSN outperforms other advanced models in speed and efficiency when dealing with large-scale spatiotemporal datasets.

6. Conclusions and future work

In this paper, we introduce a novel spatiotemporal time series forecasting model known as the MPFGSN. The core strength of this model

Table 4
The experimental time consumption (seconds per epoch) for the MPFGSN(5), MPFGSN(3), FourierGNN, MiTSformer, CycleNet, and PatchTST models on the ETTm1, ECG, exchange_rate, and CSI300 datasets. The best results are in bold and the second best are underlined.

Model		MPFGSN(5)		MPFGSN(3)		FourierGNN		MiTSformer		CycleNet		PatchTST	
Metric		MSE	Time	MSE	Time	MSE	Time	MSE	Time	MSE	Time	MSE	Time
ETTm1	96	**0.3635**	668.6841	0.3865	**101.5455**	0.4633	168.4347	0.4151	1446.2013	0.3670	158.7039	0.3752	120.9934
	192	**0.4273**	671.0116	0.4501	**105.2163**	0.5187	131.0848	0.4838	1469.4564	0.4382	153.9528	0.4453	121.6779
	336	**0.4864**	679.8428	0.5087	**110.6427**	0.6335	139.0116	0.5444	1403.2789	0.5087	157.2829	0.5162	119.3538
	720	**0.5483**	724.0919	0.5755	134.1837	0.6943	160.2650	0.6054	1442.0798	0.5823	150.0121	0.5881	**117.7702**
ECG	48	**1.0491**	89.7300	1.1047	**11.9400**	1.1287	13.4400	1.0675	246.8558	1.0946	12.0400	1.1815	12.2300
	96	**1.1343**	90.0800	1.1893	**11.3500**	1.2036	13.7500	1.1710	248.0204	1.1804	11.5886	1.2682	12.2760
	192	**1.3116**	125.2300	1.3216	**11.1200**	1.3336	13.1200	1.3823	243.1830	1.3467	11.8737	1.4302	12.3809
	336	**1.4105**	126.3000	1.4301	11.6900	1.4353	12.1500	1.5811	225.4713	1.5085	**11.3846**	1.5953	12.1632
exchange_rate	48	0.0530	90.4600	0.0545	**10.5200**	0.0563	22.5700	**0.0474**	154.0224	0.0550	15.3179	0.0567	17.4676
	96	**0.0856**	95.9700	0.0995	**10.1300**	0.1644	22.2100	0.1148	143.1229	0.0889	15.7117	0.1005	16.6838
	192	**0.1730**	96.4100	0.2345	**10.2000**	1.2405	21.8500	0.1857	148.6874	0.1798	15.5884	0.1977	16.8257
	336	**0.3033**	90.9000	0.4110	**9.9500**	1.1836	21.2800	0.3921	143.6231	0.3310	15.5740	0.3515	15.9335
CSI300	36	**0.2480**	56.3100	0.2627	**5.7100**	0.3860	6.1200	0.2574	106.9016	0.2613	7.4333	0.3024	7.1164
	48	0.3578	57.0700	0.3868	**5.7300**	0.3944	6.2000	0.3401	108.4560	**0.2995**	6.8476	0.3389	7.1912
	60	0.5308	42.2700	0.5707	**5.3000**	0.7114	6.5800	0.5111	106.3872	**0.3361**	6.5918	0.3770	6.9679
	96	0.6551	55.4900	0.8073	6.5400	1.2628	6.5400	0.6084	104.4621	**0.4266**	**6.2871**	0.4279	6.9929

lies in its ability to capture dynamic change patterns in time series by decomposing complex time series data into multiple small, continuous time segments, known as patches. This approach not only enhances the model's perception of local features in time series but also effectively integrates these local features into the model through a specially designed hyperpatch graph structure, providing a new perspective for time series analysis.

We further theoretically prove that the operations performed by the FGSN used in the MPFGSN model in the spectral domain are equivalent to multi-layer graph convolutions in the spatiotemporal domain. This equivalence not only validates the theoretical foundation of FGSN but also demonstrates, through complexity analysis, that FGSN significantly outperforms traditional methods in terms of computational efficiency, which is particularly important for processing large-scale time series data.

Another key feature of the MPFGSN model is its ability to handle patches at different resolution levels and integrate information from these levels through a residual fusion mechanism. This mechanism not only improves the model's ability to capture details in time series data but also enhances the model's predictive accuracy for long-term trends and short-term fluctuations in time series by eliminating information redundancy.

Through extensive experiments on multiple public time series datasets, we have demonstrated that the MPFGSN model surpasses existing time series forecasting methods on various evaluation metrics. These experimental results not only highlight the theoretical advantages of the MPFGSN model but also showcase its potential in practical applications, especially in scenarios requiring the handling of data with complex spatiotemporal dependencies.

In the future, we will explore two directions to further investigate the potential applications of our model: firstly, applying our model to other time series analyses, such as classification and anomaly detection; secondly, integrating self-supervised learning to enhance the model's capabilities in unsupervised learning.

CRediT authorship contribution statement

Jiankai Zheng: Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Conceptualization. **Liang Xie:** Writing – review & editing, Supervision, Conceptualization. **Haijiao Xu:** Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the Natural Science Foundation of Guangdong Province, China [grant number 2020A1515011208], Science and Technology Program of Guangzhou [grant number 20210208 0353], Characteristic Innovation Project of Guangdong Province [grant number 2019KTSCX117], and PCL-CMCC Foundation for Science and Innovation (Grant No. 2024ZY2B0050). We thank all the anonymous reviewers for their generous contributions of time and effort. Their expert advice has significantly elevated the quality of our manuscript.

Appendix A. Explanations and proofs

Lemma 1.

$$\mathcal{F}_G(X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i) = \mathcal{F}_G(X) \cdot \prod_{j=1}^{i} S_j, S_j = \mathcal{F}_G(\kappa_j) \tag{25}$$

where $\mathcal{F}_G$ denotes the Graph Spectral Transform (GST).

Proof. Given $\mathcal{F}_G(X *_G \kappa) = \mathcal{F}_G(x) \cdot S_\kappa$, we can expand the multi-layer convolutions $X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i$ as follows:

$$\begin{aligned}
&\mathcal{F}_G(X \cdot *_G \kappa_1 \cdot *_G \kappa_2 \cdot *_G \cdots *_G \kappa_i) \\
&= \mathcal{F}_G(X \cdot *_G \kappa_1 \cdot *_G \kappa_2 \cdot *_G \cdots *_G \kappa_{i-1}) \cdot \mathcal{F}_G(\kappa_i) \\
&= \mathcal{F}_G(X \cdot *_G \kappa_1 \cdot *_G \kappa_2 \cdot *_G \cdots *_G \kappa_{i-1}) \cdot S_i \\
&= \mathcal{F}_G(X \cdot *_G \kappa_1 \cdot *_G \kappa_2 \cdot *_G \cdots *_G \kappa_{i-2}) \cdot \mathcal{F}_G(\kappa_{i-1}) \cdot S_i \\
&\vdots \\
&= \mathcal{F}_G(X) \cdot S_0 \cdot S_1 \cdots S_{i-1} \cdot S_i \\
&= \mathcal{F}_G(X) \cdot \prod_{j=0}^{i} S_j
\end{aligned} \tag{26}$$

Thus, $\mathcal{F}_G{}^{-1}(\mathcal{F}_G(X) \cdot \prod_{i=0}^{k_i} S_i) = X *_G \kappa_1 *_G \kappa_2 *_G \cdots *_G \kappa_i$. □

Table B.5
Summary of results for all methods on nine datasets. The best results are in bold and the second best are underlined.

Model		MPFGSN	CycleNet	MiTSformer	iTransformer	PatchTST	FourierGNN	TimesNet	DLinear	Crossformer
Metric		MAPE	MAPE	MAPE	MAPE	MAPE	MAPE	MAPE	MAPE	MAPE
ETTm1	96	1.9091	**1.8122**	1.9811	2.1133	2.1209	2.2211	2.6249	1.9300	1.9500
	192	2.0697	**1.9611**	2.1904	2.3042	2.2742	2.2942	2.6797	2.0185	2.0431
	336	2.1637	2.1159	2.3716	2.4983	2.4429	2.3142	2.9843	2.0849	**1.9296**
	720	2.2523	2.3417	2.5994	2.7983	2.6926	2.4667	3.2036	2.1757	**2.0691**
ETTm2	96	**1.3150**	1.5884	2.2161	1.8504	1.8313	1.6688	2.1942	1.8712	2.1891
	192	**1.3609**	1.7507	2.3071	1.9076	1.9033	1.8685	2.2580	1.8566	2.2361
	336	**1.3633**	1.9638	2.4204	1.9836	1.9873	2.0999	2.2617	1.8052	2.2494
	720	**1.4921**	2.2280	2.3797	2.0376	2.0393	2.7495	2.2251	1.7558	2.4279
ETTh1	96	2.0399	2.7741	2.6874	2.4266	2.4846	2.2787	2.9513	**1.9208**	2.4279
	192	2.0766	2.9535	2.8493	2.5971	2.5744	2.3162	3.2579	1.8863	**1.8162**
	336	2.0902	3.1253	2.8277	2.6837	2.6168	2.3692	2.9156	**1.8466**	1.8993
	720	2.2341	3.7007	2.9935	2.9257	2.8489	2.4366	3.1640	**1.8878**	2.1902
ETTh2	96	**1.4481**	2.0657	2.3811	2.0841	2.0871	1.4765	2.3458	2.0185	1.9761
	192	**1.5503**	2.2870	2.1622	2.1689	2.1655	1.5597	2.4113	2.0163	2.0959
	336	**1.5929**	2.4866	1.9302	2.2177	2.1920	1.6108	2.3067	1.9371	1.9672
	720	**1.4955**	2.9885	1.9904	2.2311	2.2022	1.8908	2.4019	1.8653	2.0270
ECG	48	3.2291	3.5710	5.0878	4.7299	4.5906	**2.7484**	4.9493	3.4607	3.5764
	96	3.3312	3.7706	5.3071	4.8755	4.7879	**2.8762**	5.1554	3.5806	3.9851
	192	3.7478	4.3029	5.9232	5.5029	5.3802	**2.9915**	5.7277	4.0775	4.2754
	336	4.4936	5.4556	7.2994	6.6398	6.4965	**3.2268**	6.8508	5.1366	5.3132
exchange_rate	48	0.9351	**0.8923**	0.9819	1.1500	1.2056	0.9702	1.4079	1.2940	1.2094
	96	**1.2327**	1.2642	1.2446	1.4868	1.5221	1.4155	1.6692	1.5604	1.3902
	192	1.8632	1.9118	1.7434	2.2045	2.2128	**1.6992**	2.4184	2.1694	2.1388
	336	2.3609	2.9759	2.3327	3.2118	**2.2128**	2.2641	3.4218	2.9532	2.7944
PeMS07	96	**2.4363**	3.8715	4.0139	4.2705	4.4073	2.7313	5.0842	2.6680	3.4602
	192	2.6600	3.8260	4.4401	4.4662	4.6014	2.8222	5.4005	2.8300	**2.0056**
	336	2.7324	3.7041	4.3478	4.4459	4.4088	3.0601	5.2645	3.0012	**1.5703**
	720	2.7483	3.7913	4.5051	4.6124	4.5542	3.0510	5.3076	3.0103	**1.2905**
CSI300	36	**1.4192**	1.6048	2.5163	2.0717	2.1470	1.7236	2.2929	2.2110	2.9291
	48	**1.6349**	1.7405	2.5404	2.2146	2.2722	1.7883	2.4300	2.3191	3.3522
	60	**1.8101**	1.8750	2.7930	2.3629	2.4045	1.8820	2.5709	2.4159	3.7969
	96	**1.9529**	2.2929	3.3390	2.7837	2.8142	2.2337	2.9592	2.7700	4.3669
METR-LA	96	1.6494	2.1215	2.6091	2.5080	2.3863	1.7074	2.8784	1.6506	**1.6071**
	192	1.6688	2.1695	2.8627	2.6446	2.4264	1.9277	3.0013	1.6047	**1.5937**
	336	1.6837	2.2663	2.9557	2.7399	2.5359	1.5411	3.0439	**1.5214**	1.6836
	720	1.5350	2.4448	3.4167	3.1229	2.9489	1.3750	3.3120	1.3632	**1.3295**
Avg rank		**2.0556**	4.6944	6.6944	6.2778	5.8889	3.4444	8.3056	3.3889	4.2500

Appendix B. MAPE results of comparative experiments

B.1. Evaluation metrics

We employ Mean Absolute Percentage Error (MAPE) as the evaluation metric, which is defined as follows:

$$MAPE = \frac{1}{\mathcal{T}_{Pred} N} \sum_{i=1}^{\mathcal{T}_{Pred}} \sum_{j=1}^{N} \left| \frac{Y_{test}^{i,j} - \hat{Y}_{test}^{i,j}}{Y_{test}^{i,j}} \right| \tag{27}$$

where $\mathcal{T}_{Pred}$ is the number of prediction time steps, N is the data dimension, $Y_{test}^{i,j}$ represents the true value from the test set at the ith time step and jth dimension, and $\hat{Y}_{test}^{i,j}$ is the corresponding predicted value for the test set at the same time step and dimension.

B.2. Experimental results

The experimental results with the MAPE as the evaluation metric are shown in Table B.5. The rankings of the comparative models have changed to some extent, while the MPFGSN model still performs outstandingly, with an average ranking of 2.0556. Especially on the ETTm2, ETTh2, and CSI300 datasets, this model outperforms other comparative methods at all prediction window lengths, demonstrating good adaptability and prediction accuracy.

Data availability

Data will be made available on request.

References

[1] S. Liao, L. Xie, Y. Du, S. Chen, H. Wan, H. Xu, Stock trend prediction based on dynamic hypergraph spatio-temporal network, Appl. Soft Comput. 154 (2024) 111329.

[2] H. Xia, H. Ao, L. Li, Y. Liu, S. Liu, G. Ye, H. Chai, CI-STHPAN: Pre-trained attention network for stock selection with channel-independent spatio-temporal hypergraph, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024, pp. 9187–9195, 8.

[3] H. Su, X. Wang, Y. Qin, Q. Chen, Attention based adaptive spatial–temporal hypergraph convolutional networks for stock price trend prediction, Expert Syst. Appl. 238 (2024) 121899.

[4] S. Bai, J.Z. Kolter, V. Koltun, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018, arXiv preprint arXiv:1803.01271.

[5] D. Salinas, V. Flunkert, J. Gasthaus, T. Januschowski, DeepAR: Probabilistic forecasting with autoregressive recurrent networks, Int. J. Forecast. 36 (3) (2020) 1181–1191.
[6] J. Chen, C. Zhao, Addressing spatial-temporal heterogeneity: General mixed time series analysis via latent continuity recovery and alignment, in: Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 17910–17946.
[7] S. Lin, W. Lin, X. HU, W. Wu, R. Mo, H. Zhong, CycleNet: Enhancing time series forecasting through modeling periodic patterns, in: Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 106315–106345.
[8] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, Q. Zhang, Spectral temporal graph neural network for multivariate time-series forecasting, in: Advances in Neural Information Processing Systems, vol. 33, 2020.
[9] L. BAI, L. Yao, C. Li, X. Wang, C. Wang, Adaptive graph convolutional recurrent network for traffic forecasting, in: Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 17804–17815.
[10] Y. Chen, I. Segovia-Dominguez, B. Coskunuzer, Y. Gel, TAMP-S2GCNets: Coupling time-aware multipersistence knowledge representation with spatio-supra graph convolutional networks for time-series forecasting, in: International Conference on Learning Representations, 2022.
[11] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2017.
[12] C. Cui, X. Li, C. Zhang, W. Guan, M. Wang, Temporal-relational hypergraph tri-attention networks for stock trend prediction, Pattern Recognit. 143 (2023) 109759.
[13] K. Yi, Q. Zhang, W. Fan, H. He, L. Hu, P. Wang, N. An, L. Cao, Z. Niu, FourierGNN: Rethinking multivariate time series forecasting from a pure graph perspective, in: Advances in Neural Information Processing Systems, vol. 36, 2023, pp. 69638–69660.
[14] F. Feng, H. Chen, X. He, J. Ding, M. Sun, T.-S. Chua, Enhancing stock movement prediction with adversarial training., in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, vol. 19, 2019, pp. 5843–5849.
[15] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, G.W. Cottrell, A dual-stage attention-based recurrent neural network for time series prediction, in: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, 2017, pp. 2627–2633.
[16] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, C. Zhang, Connecting the dots: Multivariate time series forecasting with graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 753–763.
[17] M. Hou, C. Xu, Z. Li, Y. Liu, W. Liu, E. Chen, J. Bian, Multi-granularity residual learning with confidence estimation for time series prediction, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 112–121.
[18] M.S. Reis, Multiscale and multi-granularity process analytics: A review, Processes 7 (2) (2019) 61.
[19] Y. Zhang, L. Ma, S. Pal, Y. Zhang, M. Coates, Multi-resolution time-series transformer for long-term forecasting, in: International Conference on Artificial Intelligence and Statistics, 2024, pp. 4222–4230.
[20] S. Zhong, S. Song, W. Zhuo, G. Li, Y. Liu, S.-H.G. Chan, A multi-scale decomposition MLP-mixer for time series analysis, Proc. VLDB Endow. 17 (7) (2024) 1723–1736.
[21] Y. Zhang, J. Yan, Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: International Conference on Learning Representations, 2022.
[22] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: International Conference on Learning Representations, 2023.
[23] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics, 2019, pp. 4171–4186.
[24] Y. Zhang, L. Ma, S. Pal, Y. Zhang, M. Coates, Multi-resolution time-series transformer for long-term forecasting, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2024, pp. 4222–4230.
[25] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, in: International Conference on Learning Representations, 2022.
[26] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J.Y. Zhang, J. Zhou, Timemixer: Decomposable multiscale mixing for time series forecasting, 2024, arXiv preprint arXiv:2405.14616.
[27] R. Gao, J. Wang, Y. Yu, J. Wu, L. Zhang, Enhanced graph diffusion learning with dynamic transformer for anomaly detection in multivariate time series, Neurocomputing 619 (2025) 129168.
[28] B. Jing, D. Zhou, K. Ren, C. Yang, Causality-aware spatiotemporal graph neural networks for spatiotemporal time series imputation, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, 2024, pp. 1027–1037.
[29] Z. Dong, R. Jiang, H. Gao, H. Liu, J. Deng, Q. Wen, X. Song, Heterogeneity-informed meta-parameter learning for spatiotemporal time series forecasting, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 631–641.
[30] W. Kong, Z. Guo, Y. Liu, Spatio-temporal pivotal graph neural networks for traffic flow forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024, pp. 8627–8635, 8.
[31] Z. Li, L. Xia, Y. Xu, C. Huang, GPT-ST: Generative pre-training of spatio-temporal graph neural networks, in: Advances in Neural Information Processing Systems, vol. 36, 2023, pp. 70229–70246.
[32] M. Schuster, K. Nakajima, Japanese and korean voice search, in: 2012 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, 2012, pp. 5149–5152.
[33] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations, 2021.
[34] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, 2018, pp. 3634–3640.
[35] Y. Li, R. Yu, C. Shahabi, Y. Liu, Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, in: International Conference on Learning Representations, 2018.
[36] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang, Graph wavenet for deep spatial-temporal graph modeling, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, 2019, pp. 1907–1913.
[37] G. Mateos, S. Segarra, A.G. Marques, A. Ribeiro, Connecting the dots: Identifying network structure via graph signal processing, IEEE Signal Process. Mag. 36 (3) (2019) 16–43.
[38] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, 2017, pp. 1263–1272.
[39] J. Bruna, W. Zaremba, A. Szlam, Y. LeCun, Spectral networks and deep locally connected networks on graphs, in: International Conference on Learning Representations, 2014.
[40] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, in: Advances in Neural Information Processing Systems, vol. 29, 2016, pp. 3837–3845.
[41] Y. Katznelson, An Introduction to Harmonic Analysis, Cambridge University Press, 2004.
[42] B. Ricaud, P. Borgnat, N. Tremblay, P. Gonçalves, P. Vandergheynst, Fourier could be a data scientist: From graph Fourier transform to signal processing on graphs, C. R. Phys. 20 (5) (2019) 474–488.
[43] D.I. Shuman, B. Ricaud, P. Vandergheynst, Vertex-frequency analysis on graphs, Appl. Comput. Harmon. Anal. 40 (2) (2016) 260–291.
[44] H.A. Dau, A. Bagnall, K. Kamgar, C.-C.M. Yeh, Y. Zhu, S. Gharghabi, C.A. Ratanamahatana, E. Keogh, The UCR time series archive, IEEE/ CAA J. Autom. Sin. 6 (6) (2019) 1293–1305.
[45] G. Lai, W.-C. Chang, Y. Yang, H. Liu, Modeling long-and short-term temporal patterns with deep neural networks, in: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, 2018, pp. 95–104.
[46] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, Z. Jia, Freeway performance measurement system: Mining loop detector data, Transp. Res. Rec. 1748 (1) (2001) 96–102.
[47] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, Itransformer: Inverted transformers are effective for time series forecasting, in: International Conference on Learning Representations, 2024.
[48] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting? in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, 2023, pp. 11121–11128, 9.

JianKai Zheng received the B.S. degree from Wuhan University of Technology, China, in 2024. He is currently pursuing the master's degree with the School of Mathematics and Statistics, Wuhan University of Technology, China. His current research interests include time series forecasting.

Liang Xie received the B.S. degree from Wuhan University of Technology, China, in 2009, the Ph.D. degree from Huazhong University of Science and Technology, China, in 2015. He is currently an Associate Professor in the School of Mathematics and Statistics at Wuhan University of Technology. His current research interests include time series forecasting and graph learning.

Haijiao Xu received his Ph.D. degree in computer science and technology from Huazhong University of Science and Technology, Wuhan in 2015. His research interests include deep neural network, data mining, and stock trend forecasting.